

Assignment-05 (Lab-5.5)

Course : AI Assisted Coding

Topic:- Ethical Foundations – Responsible AI Coding Practices

Student Details:

Name : Batti Chandan Singh

Hall Ticket No : 2303A51515

Batch : 22

Task-01

Final Optimal Prompt

I want you to generate two different Python solutions for checking whether a number is prime. The first should be a simple, naive approach, and the second should be an optimized approach that uses the square-root method.

Please provide clear comments and also explain the time complexity of both methods so that I can understand how efficiency improves.

The code should be clean, readable, and suitable for teaching algorithm transparency

Code Screenshot

```
Assignment-05(Lab-5.5).py > ...
1  #----Task-01/Lab-5.5.py----
2  '''I want you to generate two different Python solutions for checking whether a number is
3  The first should be a simple, naive approach, and the second should be an optimized approach
4  Please provide clear comments and also explain the time complexity of both methods so that
5  The code should be clean, readable, and suitable for teaching algorithm transparency'''
6  def is_prime_naive(n):
7      """
8      Check if a number is prime using a naive approach.
9      Time complexity: O(n)"""
10     if n <= 1:
11         return False
12     for i in range(2, n):
13         if n % i == 0:
14             return False
15     return True
16
17  def is_prime_optimized(n):
18      """
19      Check if a number is prime using an optimized approach (square-root method).
20      Time complexity: O(√n)"""
21     if n <= 1:
22         return False
23     for i in range(2, int(n**0.5) + 1):
24         if n % i == 0:
25             return False
26     return True
27
28  # Example usage:
29  number = int(input("Enter a number to check if it's prime: "))
30  print(f"Naive approach: Is {number} prime? {is_prime_naive(number)}")
31  print(f"Optimized approach: Is {number} prime? {is_prime_optimized(number)}")
32
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\py  
Coding/Assignment-05(Lab-5.5).py"  
Enter a number to check if it's prime: 5  
Naive approach: Is 5 prime? True  
Optimized approach: Is 5 prime? True  
PS D:\6th Sem 2025-26\AI-Assisted Coding> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

Zero-shot prompting refers to giving an AI model a task **without providing any examples**. The model must rely entirely on its general knowledge and understanding of language. This method tests how well the AI can interpret instructions that are new or unfamiliar. It's useful for quick problem-solving when we don't want to prepare demonstrations or patterns. However, the output may vary depending on how clearly the prompt is written. Overall, zero-shot prompting highlights the model's ability to generalize and handle tasks using only the information provided at that moment.

Task-02

Final Optimal Prompt

Generate a Python function that calculates Fibonacci numbers using recursion.

I want very clear comments that explain the base cases and how recursion works step by step.

Write the explanation in a way that helps beginners understand what happens during each recursive call.

Please make sure the comments accurately match the function's execution so the logic is transparent.

Code Screenshot

```
#---Task-02//Lab-5.5.py---
'''Generate a Python function that calculates Fibonacci numbers using recursion.
I want very clear comments that explain the base cases and how recursion works step by step.
Write the explanation in a way that helps beginners understand what happens during each recursive call.
Please make sure the comments accurately match the function's execution so the logic is transparent.'''

def fibonacci_recursive(n):
    if n < 0:
        raise ValueError("Input should be a non-negative integer.")
    # Base case: the first Fibonacci number is 0
    if n == 0:
        return 0
    # Base case: the second Fibonacci number is 1
    elif n == 1:
        return 1
    else:
        # Recursive case: Fibonacci(n) = Fibonacci(n-1) + Fibonacci(n-2)
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

# Example usage:
num = int(input("Enter a non-negative integer to find its Fibonacci number: "))
print(f"The {num}th Fibonacci number is: {fibonacci_recursive(num)}")
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.13.6\python.exe Coding/demo.py"
Enter a non-negative integer to find its Fibonacci number: 5
The 5th Fibonacci number is: 5
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.13.6\python.exe Coding/demo.py"
Enter a non-negative integer to find its Fibonacci number: 6
The 6th Fibonacci number is: 8
PS D:\6th Sem 2025-26\AI-Assisted Coding> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

One-shot prompting improves AI reliability by giving the model **one example** before assigning the actual task. This example acts as a guide that teaches the AI the expected structure, tone, or style. By seeing a single demonstration, the model can make better predictions and follow patterns more accurately. One-shot prompts are helpful when consistency matters, such as formatting or generating specific types of answers. Although still simpler than few-shot prompting, it strikes a balance between flexibility and accuracy. In many cases, one example is enough to significantly improve the model's understanding.

Task-03

Final Optimal Prompt

Write a Python program that reads a file and processes its contents. I want proper error handling for all common exceptions—such as file not found, permission denied, and decoding errors. Add comments explaining why each exception is handled and what scenario it covers.

The goal is to demonstrate transparency in error handling so the user understands exactly what can go wrong and how the program responds.

Code Screenshot

```
#---Task-03//Lab-5.5.py---
'''Write a Python program that reads a file and processes its contents. I want proper error
and decoding errors. Add comments explaining why each exception is handled and what scenario
The goal is to demonstrate transparency in error handling so the user understands exactly v
def read_file(file_path):
    try:
        # Attempt to open the file
        with open(file_path, 'r', encoding='utf-8') as file:
            content = file.read()
            print("File content successfully read.")
            return content
    except FileNotFoundError:
        # This exception is raised when the specified file does not exist
        print(f"Error: The file '{file_path}' was not found. Please check the file path and
    except PermissionError:
        # This exception is raised when the program does not have permission to access the
        print(f"Error: Permission denied for accessing the file '{file_path}'. Please check
    except UnicodeDecodeError:
        # This exception is raised when there is an error decoding the file content
        print(f"Error: Could not decode the contents of the file '{file_path}'. It may not
    except Exception as e:
        # Catch-all for any other exceptions that may occur
        print(f"An unexpected error occurred: {e}")
# Example usage:
file_path = input("Enter the path of the file to read: ")
read_file(file_path)
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.13.6\python.exe" "d:/6th Sem
Coding/demo.py"
Enter the path of the file to read: D:\6th Sem 2025-26\AI-Assisted Coding\Lab-03.py
File content successfully read.
PS D:\6th Sem 2025-26\AI-Assisted Coding> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task compares zero-shot and one-shot prompting to understand how examples influence AI performance. Zero-shot relies purely on instructions, while one-shot uses a single example to shape the model's output. The comparison helps identify when examples

make the results more accurate or structured. It also highlights situations where examples are unnecessary and instructions alone are sufficient. By analyzing both approaches, we learn how prompt design affects clarity and consistency. This understanding helps decide which method is more suitable for different use-cases in real applications.

Task-04

Final Optimal Prompt

Create a Python-based login system, but I want you to analyze the security of whatever code you generate. First, generate the simple version that Copilot normally writes (even if insecure).

Then explain the weaknesses—like plain-text passwords—and produce a revised secure version using password hashing and proper input validation.

Include a short explanation of the best practices for safe authentication because I want to understand security concerns from a developer's perspective

Code Screenshot

```
#----Task-04//Lab-5.5.py----
'''Create a Python-based login system, but I want you to analyze the security of whatever code you generate. First, generate the simple version that Copilot normally writes (even if insecure). Then explain the weaknesses—like plain-text passwords—and produce a revised secure version using password hashing and proper input validation. Include a short explanation of the best practices for safe authentication because I want to understand security concerns from a developer's perspective'''
import hashlib

# Simple (Insecure) Login System
users_db = {'user1': 'password123', 'user2': 'mypassword'}
def login_simple(username, password):
    """Simple login function that checks plain-text passwords."""
    if username in users_db and users_db[username] == password:
        return "Login successful!"
    else:
        return "Login failed!"

# Weaknesses:
# 1. Plain-text passwords: Storing passwords in plain text is a major security risk
# 2. No input validation: The function does not validate inputs, making it vulnerable to injection attacks
# 3. No account lockout: There is no mechanism to prevent brute-force attacks

# Secure Login System
secure_users_db = {
    'user1': hashlib.sha256('password123'.encode()).hexdigest(),
    'user2': hashlib.sha256('mypassword'.encode()).hexdigest() }
def login_secure(username, password):
    """Secure login function that uses hashed passwords."""
    hashed_password = hashlib.sha256(password.encode()).hexdigest()
    if username in secure_users_db and secure_users_db[username] == hashed_password:
        return "Login successful!"
    else:
        return "Login failed!"

username = input("Enter username: ")
password = input("Enter password: ")
print(login_secure(username, password))
```

Output Screenshot

```
Coding/demo.py"
Enter username: user1
Enter password: password123
Login successful!
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:
Coding/demo.py"
Enter username: user2
Enter password: mypassword
Login successful!
PS D:\6th Sem 2025-26\AI-Assisted Coding> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task evaluates the difference between procedural code and modular code using functions. Procedural code runs step-by-step without organizing logic into reusable blocks, making it simple but harder to manage in large systems. Modular code uses functions to separate responsibilities, improving readability and reusability. Comparing both helps us understand how functions reduce repetition and simplify debugging. It also shows how modular design supports scalability and long-term maintenance. In real-world projects, modular coding is preferred because it is cleaner, flexible, and easier to extend.

Tack-05

Final Optimal Prompt

Generate a Python script that logs user activity such as username, IP address, and timestamp. But I want you to analyze it for privacy risks and point out any sensitive data that should not be stored.

After identifying the issues, produce an improved version that masks or anonymizes sensitive information. Add a brief explanation of privacy-aware logging principles so I understand how to apply this responsibly.

Code Screenshot

```

#---Task-05//Lab-5.5.py---
'''Generate a Python script that logs user activity such as username, IP address, and time.
But I want you to analyze it for privacy risks and point out any sensitive data that should be masked.
After identifying the issues, produce an improved version that masks or anonymizes sensitive data.
Add a brief explanation of privacy-aware logging principles so I understand how to apply them.'''

import logging
from datetime import datetime

# Configure logging
logging.basicConfig(filename='user_activity.log', level=logging.INFO, format='%(asctime)s %(message)s')

def log_user_activity(username, ip_address):
    """Logs user activity with username, IP address, and timestamp."""
    logging.info(f'User: {username}, IP: {ip_address}')

# Privacy Risks:
# 1. Storing IP addresses can be sensitive as they can be used to track users' locations.
# 2. Storing usernames may also be sensitive depending on the context and user expectations.

# Improved Version with Anonymization
def log_user_activity_privacy_aware(username, ip_address):
    """Logs user activity with anonymized IP address."""
    # Mask the last octet of the IP address for privacy
    ip_parts = ip_address.split('.')
    if len(ip_parts) == 4:
        ip_parts[-1] = 'xxx' # Masking last octet
        masked_ip = '.'.join(ip_parts)
    else:
        masked_ip = 'unknown'
    logging.info(f'User: {username}, IP: {masked_ip}')

# Privacy-Aware Logging Principles:
# 1. Minimize data collection: Only log information that is necessary for the intended purpose.
# 2. Anonymize sensitive data: Mask or anonymize data that could identify users.
# 3. Inform users: Make users aware of what data is being logged and how it will be used.

# Example usage:
username = input("Enter username: ")
ip_address = input("Enter IP address: ")
log_user_activity_privacy_aware(username, ip_address)

```

Output Screenshot

```

PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.11\python.exe" "D:\AI-Assisted Coding\Assignment-05(Lab-5.5).py"
Enter username: chandu
Enter IP address: 192.165.1.55
PS D:\6th Sem 2025-26\AI-Assisted Coding>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task focuses on analyzing how AI responds differently in zero-shot and one-shot settings. It requires observing accuracy, structure, and clarity in the outputs generated using both prompting techniques. By reviewing the results, we learn how examples influence the AI's reasoning and consistency. This comparison helps identify cases where the zero-shot approach fails or becomes unpredictable. It also shows how one-shot improves output quality by offering a reference pattern. Overall, the task strengthens understanding of prompt engineering and builds the ability to choose the best prompting method for future problems.